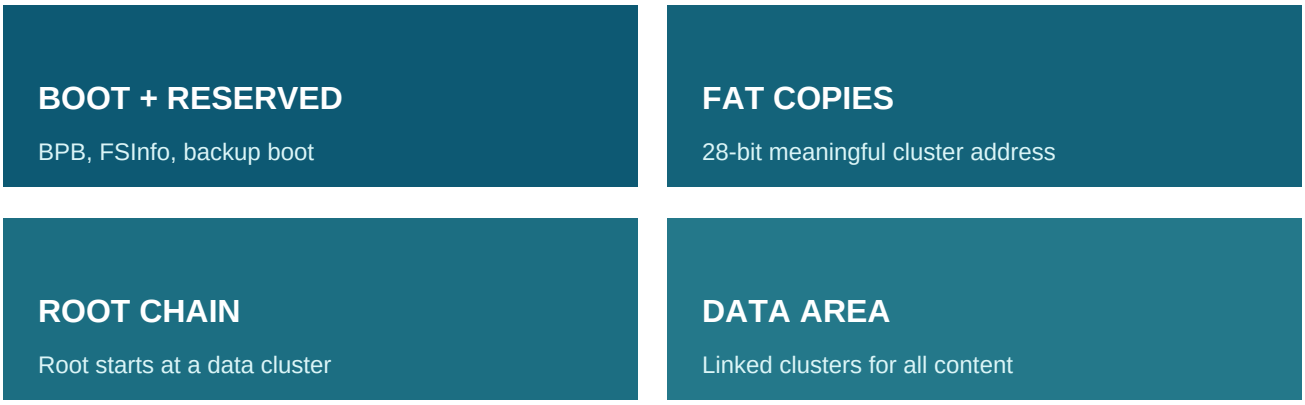
FAT32 - extended FAT volume design

ORIGINAL FORMAT / HISTORY / ENGINEERING DESIGN

FAT32 kept the chain-allocation philosophy but changed root directory and cluster numbering semantics to support much larger consumer disks.

This reference describes the original upstream disk format and its historical engineering rationale. The separate InfiltratorOS adapter is not the original specification. Historical facts, format rules and implementation limitations are separated; consult the named primary sources for field-level normative requirements.

ON-DISK STRUCTURE / CONCEPTUAL MAP



CHAPTERS

1. Historical chronology
2. Original design and on-disk structures
3. Allocation and directories
4. Integrity, validation and recovery
5. Design trade-offs and historical limitations
6. InfiltratorOS interpretation

FAT32 - extended FAT volume design

HISTORICAL CHRONOLOGY

Microsoft introduced FAT32 with Windows 95 OSR2 in 1996, answering the rising size of consumer hard disks and the slack-space penalties associated with very large FAT16 clusters. FAT32 retained FAT interoperability principles while extending the cluster-number space and removing the old fixed-size root-directory area. It later became common on removable flash media, memory cards, digital cameras, EFI System Partitions and systems that wanted a broadly understood boot filesystem.

FAT32 is a member of the same evolving family as FAT12 and FAT16, not an unrelated tree-based filesystem. exFAT was subsequently designed for large flash-oriented volumes, with a different allocation bitmap and directory-entry architecture. Modern operating systems often impose formatting or support policies that should not be confused with the actual on-disk format limits.

FAT32 - extended FAT volume design

ORIGINAL DESIGN AND ON-DISK STRUCTURES

The FAT32 BPB specifies the FAT32 sectors-per-FAT field, root cluster, FSInfo sector, boot backup location and FAT mirroring controls. Reserved sectors often include FSInfo and a backup boot sector. The root directory is now an ordinary data-cluster chain. The offset of every region must be computed from BPB geometry rather than assumed from standard formatter defaults.

Each FAT entry occupies 32 bits on disk but only the low 28 bits carry the cluster-link value. The upper four bits have reserved semantics and must be preserved when entries are modified. Data cluster numbering still begins at two. The Microsoft classification algorithm identifies FAT32 when the number of data clusters is at least 65525; text in a boot sector is not authoritative.

FAT32 - extended FAT volume design

ALLOCATION AND DIRECTORIES

The file cluster sequence is stored in a FAT chain; allocated clusters may be contiguous or scattered. The FAT32 FSInfo structure contains hints for free cluster count and a next-free search starting point, but these hints are advisory and can be stale. A robust writer must verify them against actual table contents before trusting a candidate.

Directory records retain the DOS short-name format and VFAT long-name entry sequence. Root and ordinary directories share the same general chain-based structure, which eliminates FAT16’s separate fixed root-entry capacity. DOS file attributes, starting cluster high/low halves, timestamps and size fields must be interpreted together. Despite a larger FAT namespace, file-size fields in ordinary directory entries remain 32-bit.

FAT32 - extended FAT volume design

INTEGRITY, VALIDATION AND RECOVERY

FAT32 normally provides no general-purpose metadata journal or full data checksums. Mirrored FAT copies and a backup boot sector are redundancy aids, not transaction logs. Errors may include lost chains, loops, cross-linked clusters, stale FSInfo free counts and inconsistent VFAT name sequences. Recovery must validate FAT entry masks and preserve unknown reserved fields.

FAT32 format details are extremely sensitive to boundary arithmetic. Reader and writer must check BPB sector geometry, FAT bounds, root-cluster validity and all cluster-chain links. During defragmentation, a journal provided by an external application is an application-specific safety mechanism, not a built-in feature of the Microsoft format.

FAT32 - extended FAT volume design

DESIGN TRADE-OFFS AND HISTORICAL LIMITATIONS

FAT32 traded more allocation-table space and more complex reserved-sector metadata for reduced cluster sizes on large disks. It preserved compatibility with short DOS names and removable-media firmware, but retained linear chains and the potential performance cost of severe fragmentation. Standard FAT directory entries encode a 32-bit file size, yielding the familiar under-4-GiB single-file ceiling.

FAT32 does not encode Unix permissions, native transactional snapshots, hard-link reference counts or cryptographic integrity. This simplicity is an advantage for interoperable firmware and media tools while placing the burden of durable writes and repair on operating systems and external utilities.

FAT32 - extended FAT volume design

INFILTRATOROS INTERPRETATION

FAT32.so is one selected filesystem module; FAT12.so and FAT16.so cannot substitute for it. Our ABI and defragmentation engine may implement functionality beyond Microsoft’s original standard, and such functionality must be identified as an implementation extension, never as a feature of historical FAT32.

Disk inspection should display real BPB fields, FSInfo hints, entry masks and directory-chain structure, not only a generic four-box diagram. Physical-media write safety, Linux kernel mounting and firmware boot must be qualified separately from generating this reference.

FAT32 - extended FAT volume design

PRIMARY SOURCES & DOCUMENTATION

References are included to make the historical and technical account auditable. The source documents, published format specifications and implementation references take precedence over this explanatory guide where a detail differs.

1. Microsoft FAT Specification including FAT32 BPB, FSInfo and cluster classification

https://www.scs.stanford.edu/~zyedidia/docs/_other/fat.pdf

2. Linux VFAT support documentation

https://docs.kernel.org/filesystems/vfat.html

Implementation boundary: the installed InfiltratorOS .so, the optional Linux kernel .ko, and the source-first installer are distinct products. The original filesystem design is documented above; claims about the adapter’s qualification require its own source, tests and release evidence.

FAT32 / technical detail

ON-DISK FIELD AND STRUCTURE INVENTORY

Format-specific field names and meanings are summarised below; where applicable the original specification defines exact endianness, offsets, checksums and format-revision limits.

1. BPB_FATSz32: size of one FAT, replacing the FAT16 field.
2. BPB_RootClus: first cluster of the root directory chain.
3. BPB_FSInfo: optional free count and next-free hints.
4. BPB_BkBootSec: location of an alternate boot sector.
5. FAT entry: 32-bit stored slot, but 28 significant link bits.
6. BPB_ExtFlags: active FAT/mirroring policy.
7. Directory first cluster: high and low halves of start cluster.
8. Directory size: 32-bit logical file-size field.

FAT32 / technical detail

HISTORICAL FORMAT DEVELOPMENT

Selected design milestones explain why this format acquired its present layout. Dates refer to historical format or implementation changes, not InfiltratorOS releases.

1. 1996: Windows 95 OSR2 introduces FAT32 for larger disks.

2. Late 1990s: FAT32 becomes common in consumer Windows.

3. 2000s: broad support in cameras and removable devices.

4. UEFI era: FAT32 remains an important firmware-readable system partition format.

Important historical distinction: newly introduced driver features are not necessarily changes to the original on-disk format. Read the version or feature bits rather than assuming current software support defines historic media.

FAT32 / technical detail

FORMAT VALIDATION AND RECOVERY INVARIANTS

A writable implementation must establish all of these conditions before modifying any original-format media. These checks describe the minimum class of invariants, not proof of complete recovery qualification.

1. Cluster count must be at least 65525 for format classification.

2. Preserve high reserved nibble when changing FAT entries.

3. Treat FSInfo counts as hints until validated against FAT.

4. Validate root-chain cluster and backup boot sector geometry.

5. Do not advertise files of 4 GiB or larger as ordinary FAT32 support.

Forensic principle: use a separate verified image, retain original structures, and report conflicting evidence instead of inventing a new supposedly clean filesystem state.

FAT32 / engineering detail

DISK READ PATH / ENGINEERING NOTES

1. Read the FAT32 extended BIOS Parameter Block, including RootClus, FAT size, backup boot sector and FSInfo sector locations.

2. Locate the first data sector from reserved sectors and FAT copies; the root directory lives in a cluster chain, not in a fixed pre-data root area.

3. FAT32 uses 32-bit table entries with only the low 28 bits for cluster values. Preserve or interpret the upper reserved bits according to the specification.

4. Check extended flags to determine whether FAT mirroring is enabled or an active FAT is selected; do not assume both FATs are always written in the same way.

5. Interpret FSInfo free-cluster count and next-free hint as advisory values. They can be stale and must not override an actual FAT scan.

6. Resolve directory chains and 32-byte entries using the same short-name and long-name relationship as FAT16, while honouring FAT32's root-cluster model.

7. Translate data cluster numbers relative to the first data sector; the BPB's sectors-per-cluster field controls the physical allocation unit.

8. Understand that some FAT32 structures are retained for backwards compatibility. Distinguish meaningful fields from legacy fields unused in FAT32.

FAT32 / engineering detail

VALIDATION, RECOVERY AND COMPATIBILITY

1. Validate FSInfo signatures, backup boot-record locations and reserved-sector bounds before trusting their data. Corrupt auxiliary structures need not imply corrupt file contents.

2. Check FAT entries against the computed data-cluster population and the proper bad-cluster and end-of-chain ranges; reject invalid references.

3. Follow each file and directory chain with cycle detection and an upper traversal bound; a damaged FAT link must not cause unbounded I/O.

4. Cross-check allocation, directory length and cluster traversal. Zero-length files, sparse-looking holes and orphaned chains require different treatment.

5. Do not use FSInfo alone to determine free space. Full verification requires checking the relevant FAT allocation entries.

6. A copied FAT is not an independently versioned journal. Crash recovery cannot assume that two matching tables prove directory updates completed.

7. Preserve long-filename sequences with their short-name checksums; invalid or deleted entries may otherwise produce plausible but false names.

8. When resizing, recalculate layout, cluster count, FAT capacity and reserved regions together; a mere change to the total-sector field is insufficient.